

Proto-Cuneiform Preprocessing

Description

This file enables preprocessing of cuneiform tablet transliterations as downloaded from CDLI (<https://cdli.mpiwg-berlin.mpg.de/search>). The extraction of proto-cuneiform sign codings is done with the sign lists provided by CDLI. In their syntax, repetitions of signs are indicated by a number followed by the sign type in parentheses, e.g. 6(N34) 1(N45) 3(N14) 5(N01), SZE~a (see Figure below). This is here expanded to yield individual sign tokens, e.g. N34 N34 ... N34 N45 N14 N14 N14, etc. We also include signs which are followed by '#' (broken but identifiable signs), but we exclude signs followed by '?' (questionable). We here only consider the periods "Uruk V", "Uruk IV", and "Uruk III", as these are typically seen as the proto-cuneiform periods, i.e. before genuine writing of spoken Sumerian developed.

```
knitr::include_graphics("~/Github/PaleoSigns/figures_external/tablet.png")
```



[MS 3147/2 \(P252157\)](#)

Primary Publication:

Collection: Schøyen Collection, Oslo, Norway

Museum no.: MS 3147/2

Provenience: uncertain (mod. uncertain)

Period: Uruk V (ca. 3500-3350 BC)

Object Type: tablet or envelope > tablet

Material: clay

Transliteration:

atf: lang qpc

tablet

obverse

1. 6(N34) 1(N45) 3(N14) 5(N01) , SZE~a

reverse

blank space

Figure 1: Example of a Proto-Cuneiform tablet and transliteration from the CDLI database. Citation for this entry: "MS 3147/2 Artifact Entry." 2004. Cuneiform Digital Library Initiative (CDLI). December 15, 2004. <https://cdli.ucla.edu/P252157>.

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"
## [1] "x86_64-pc-linux-gnu"
```

Load Libraries

```
library(Hmisc)

##
## Attaching package: 'Hmisc'
## The following objects are masked from 'package:base':
##
##      format.pval, units
library(stringr)
library(readr)
```

Give the package versions.

```
##      readr stringr  Hmisc
## "2.1.5" "1.5.1" "5.1-2"
```

Load Transliteration and Sign List

Loading transliterations and sign lists from CDLI (locally stored in the github repo under data/CDLI). Note that different datasets have to be loaded by choosing the period as a character string below.

```
# choose period (UrukIII, UrukIV, UrukV)
period <- "UrukIV"

# transliteration as text file with line breaks
translit.file = paste("~/Github/PaleoSigns/data/CDLI/",
                      paste(period, "/inscriptions.atf", sep = ""),
                      sep = "")
textlines <- scan(translit.file, what = "char", quote = "", comment.char = "",
                  encoding = "UTF-8", sep = "\n", skip = 0)
# load transliteration as single character string
textstring <- read_file(translit.file)
# give beginning for illustration purposes
head(textlines)

## [1] "&P000004 = CDLI Lexical 000002, ex. 051"
## [2] "#atf: lang qpc "
## [3] "@tablet "
## [4] "@obverse "
## [5] "@column 1 "
## [6] "$ beginning broken "
```

```

# load sign list
signlist.file = paste("~/Github/PaleoSigns/data/CDLI/",
                      paste(period, "/signlist.txt", sep = ""),
                      sep = "")
signlist <- scan(signlist.file, what = "char", quote = "", comment.char = "",
                encoding = "UTF-8", sep = "\n", skip = 0)
# give beginning of signlist
head(signlist)

## [1] "1(N01)" "1(N01@r)" "1(N02)" "1(N03)" "1(N04)" "1(N05)"

```

Process Transliterations

Get vector of IDs for tablets.

```

# the ampersand followed by P identifies a single tablet transliteration
tablet.id <- textlines[grep(pattern = "\\&P", textlines)]
# check length
length(tablet.id)

## [1] 1000

# remove ampersand
tablet.id <- gsub(pattern = "\\&", "", tablet.id)
# get just the ID
tablet.id <- substr(tablet.id, 1, 7)
# give beginning of tablet ID vector
head(tablet.id)

## [1] "P000004" "P000003" "P000005" "P000007" "P000006" "P000121"

```

Extract and concatenate the transliterations.

```

# get chunks of text by ampersand plus "P" (at the beginning of each tablet)
textchunks <- str_split(textstring, pattern = "\\&P")
# unlist and remove first element (empty)
textchunks <- unlist(textchunks)[-1]
# remove the header
textchunks <- gsub("\\=.*qpc", "", textchunks)

# use signlist provided by CDLI to extract only signs
# create a duplicate signlist with '#' following after the signs,
# this means the sign is partly broken but identifiable
signlist.hashtag <- paste0(signlist, '#')
# concatenate the two lists (i.e. vectors)
signlist <- c(signlist, signlist.hashtag)
# first some characters in the sign list need to be escaped
signlist <- escapeRegex(signlist)
# add white spaces after character strings in signlist
# this helps with only extracting full string matches
signlist <- paste0(signlist, ' ')
# remove 'X' as this is a placeholder for unidentified signs
signlist <- signlist[!signlist == 'X ']

# run for loop to extract signs based on sign list

```

```

# initialize vectors
signs.orig.vec <- c()
signs.expand.vec <- c()
for (i in 1:length(textchunks)){
  # note that the order of signs is here sometimes changed,
  # as it follows the order of signs in sign list
  signs.orig <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
  signs <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
  # trim white spaces around signs
  signs.orig <- trimws(signs.orig)
  signs <- trimws(signs)
  # remove the hashtag
  signs <- gsub("#", "", signs)
  # unravel the multiplication syntax to get multiple sign tokens
  try(silent = TRUE, { # if the processing fails for a certain row,
    # there will be no output for this row,
    # but the try() function allows the loop to keep running
    for (j in 1:length(signs)){
      sign.syntax <- signs[j]
      # get only number
      remainder <- sub(pattern = "\\(.*\\)", "", sign.syntax)
      # if number then multiply with sign type, else
      # leave sign syntax as is
      if (!grepl("[^0-9]", remainder)){
        # get sign.type inside round brackets
        sign.type <- gsub(".*\\((.*)\\).*", "\\1", sign.syntax)
        # repeat given the number ("remainder" from above)
        sign.tokens <- rep(sign.type, remainder)
        # collapse into character string
        sign.tokens <- paste(sign.tokens, collapse = " ")
      } else {
        sign.tokens <- sign.syntax
      }
      signs[j] <- sign.tokens
    }) # end of try() function
    # collapse the signs into one string
    signs <- paste(signs, collapse = " _ ")
    signs.orig <- paste(signs.orig, collapse = " _ ")
    signs.orig.vec <- c(signs.orig.vec, signs.orig)
    signs.expand.vec <- c(signs.expand.vec, signs)
  }
}
# compile final data frame
cunei.data <- data.frame(tablet.id, signs.orig.vec, signs.expand.vec)

```

Remove rows with empty sign codings.

```
cunei.data <- cunei.data[-which(cunei.data$signs.orig.vec == ""), ]
```

Select those with at least a given number of cuneiform signs.

```
cunei.data <- cunei.data[lengths(strsplit(cunei.data$signs.expand.vec, " ")) > 1, ]
```

Give head of final data frame for illustration purposes.

```
# head(cunei.data)
```

Safe to file.

```
write.csv(cunei.data, file = paste("~/Github/PaleoSigns/data/CDLI/",  
                                     paste(period, "/signs_clean.csv", sep = ""),  
                                     sep = ""), row.names = F)
```

Package References

```
## Harrell Jr F (2024). _Hmisc: Harrell Miscellaneous_. R package version  
## 5.1-2, <https://CRAN.R-project.org/package=Hmisc>.
```

```
## Wickham H (2023). _stringr: Simple, Consistent Wrappers for Common  
## String Operations_. R package version 1.5.1,  
## <https://CRAN.R-project.org/package=stringr>.
```

```
## Wickham H, Hester J, Bryan J (2024). _readr: Read Rectangular Text  
## Data_. R package version 2.1.5,  
## <https://CRAN.R-project.org/package=readr>.
```